

Seq2seq & Transformers

Ambient Intelligence and Domotics

Master Degree: Artificial Intelligence for Science and Technology

Gabriele Civitarese

EveryWare Lab, Department of Computer Science

Università degli Studi di Milano

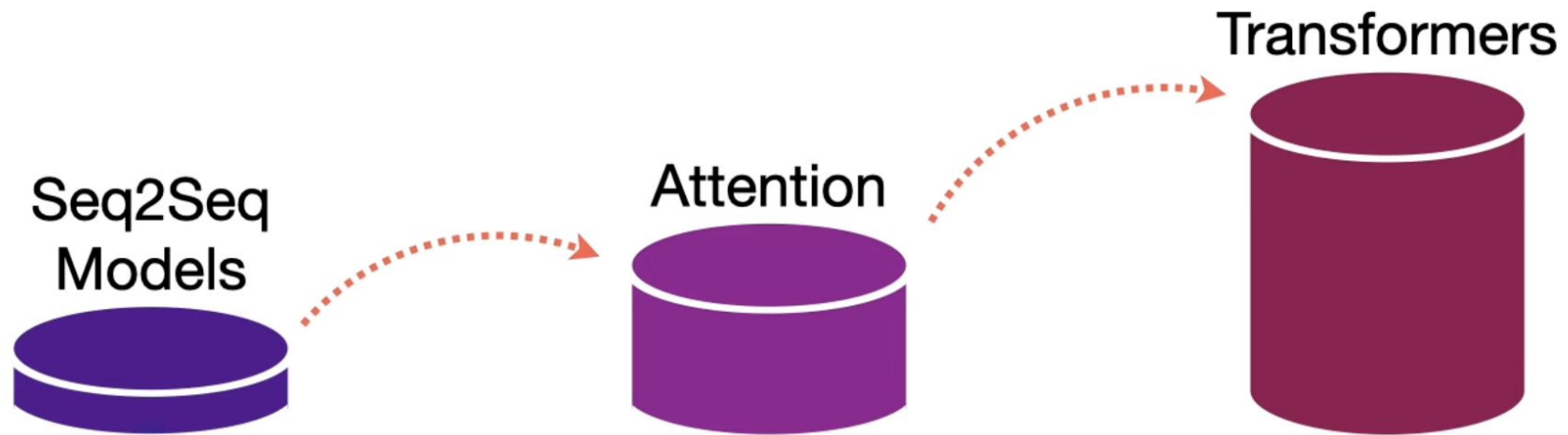
Copyright

These slides are from the teachers of this course. All the material is subject to copyright and cannot be redistributed without consent of the copyright holder. The same holds for audio and video-recordings of the classes of this course.

What we will see in this lesson?

- Previously, we studied standard deep learning architectures for time series
- They work well in many applications
 - and this also what you will use during the lab!
- However, latest advances in Natural Language Processing (NLP) introduced more sophisticated methods for sequence processing
 - ...since time series are ***sequences*** as well, we can leverage such approaches!

Plan of this lesson

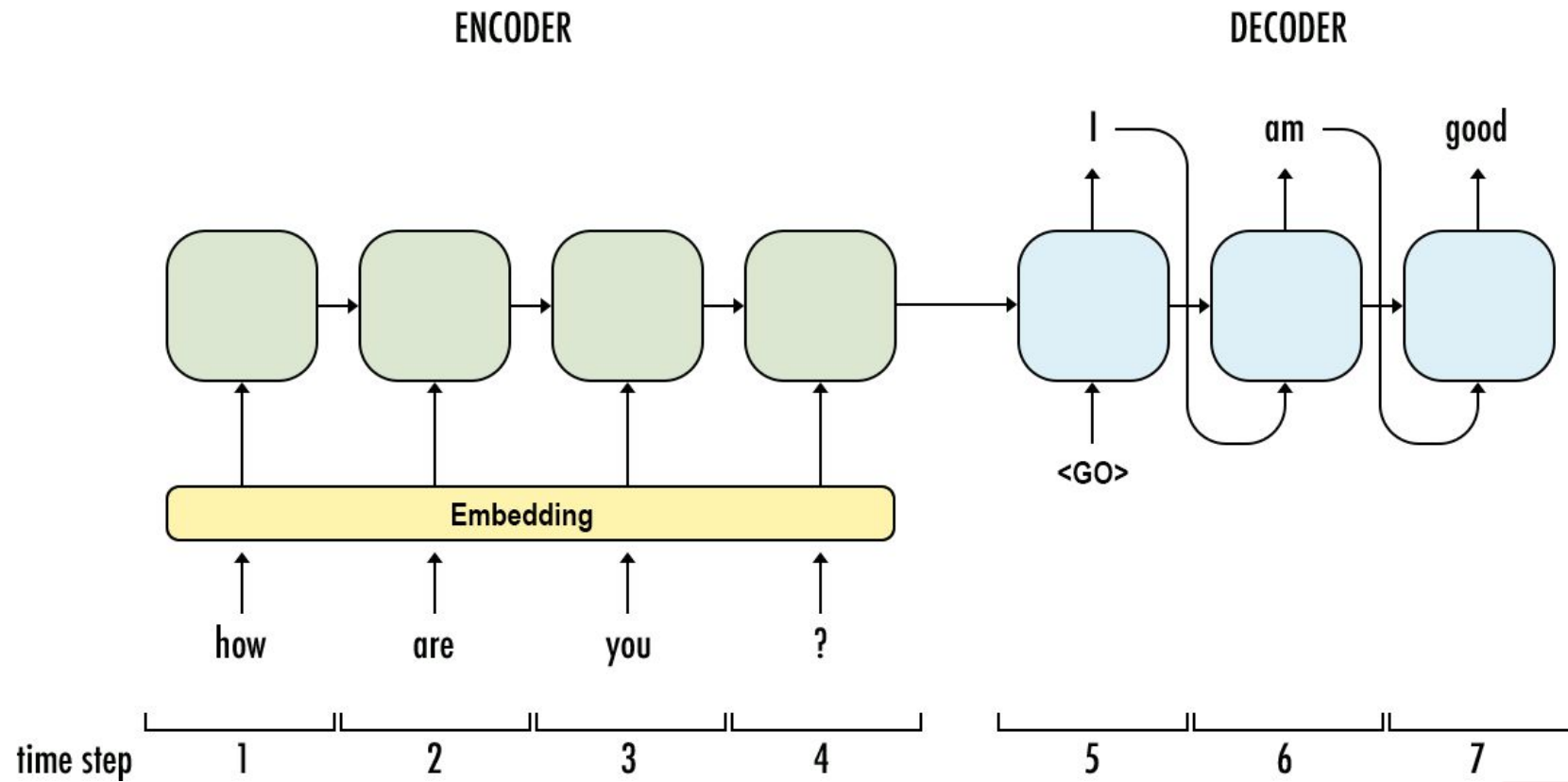


Seq2Seq Models

Seq2Seq

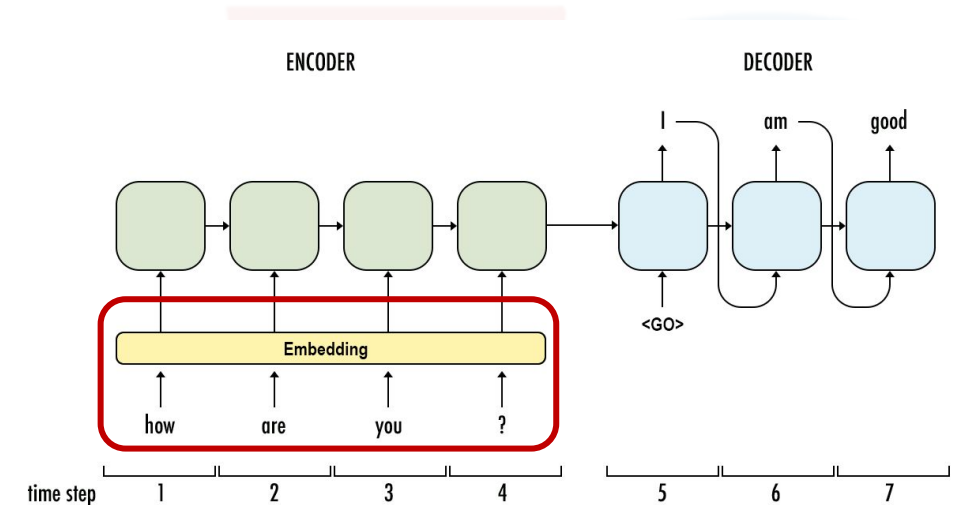
- Goal: mapping a sequence into another sequence
 - the size of input sequence may differ from size of the output sequence!
 - initially proposed for language translation in NLP
 - they can applied to time series (especially for forecasting!)
- How?
 - Using an encoder/decoder architecture
 - LSTMs are adopted to handle input/output of variable dimensions

Seq2Seq: the idea



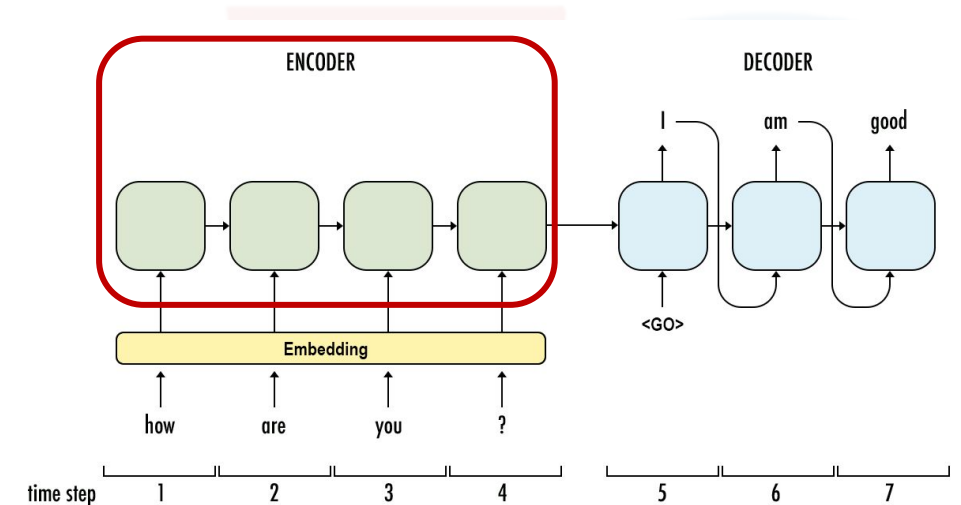
Embedding Layer

- Typically needed for Natural Language Processing (NLP) to map words into a numerical representation
 - to map a discrete input into a sequence of **tokens** (where each token is a real valued vector)
 - Hence, input and output are both sequences of **tokens**
 - There are also special tokens
 - e.g., **<EOS>** to determine when a sequence ends
- Time series may also require an embedding layer, but no special tokens
 - we will see seq2seq in a general setting before going into time series

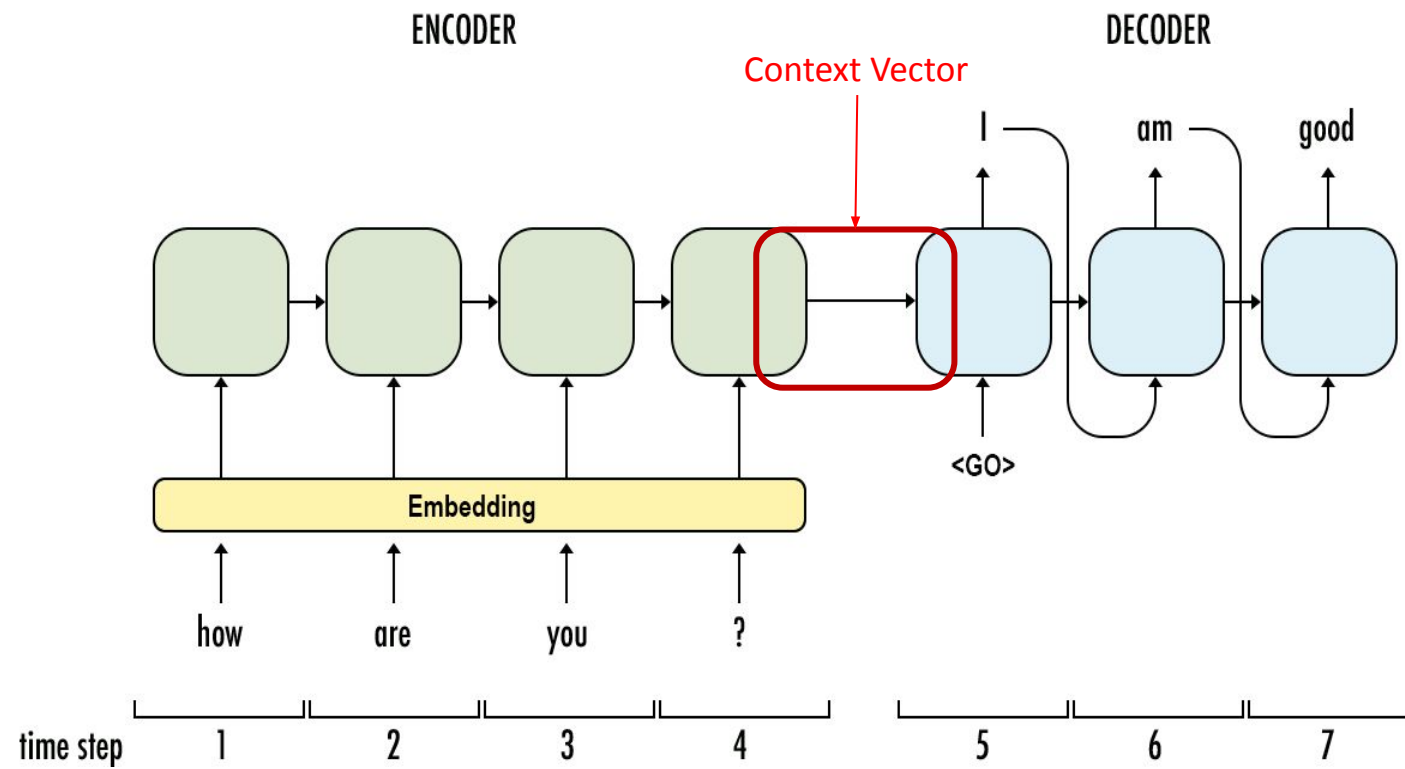


Encoder

- The Encoder part of a seq2seq model uses several LSTM layers
- Goal: mapping the input sequence into a latent representation
 - the LSTMs (as we learned in the other lesson) process one element of the sequence at a time
 - this latent representation is also called ***context vector***

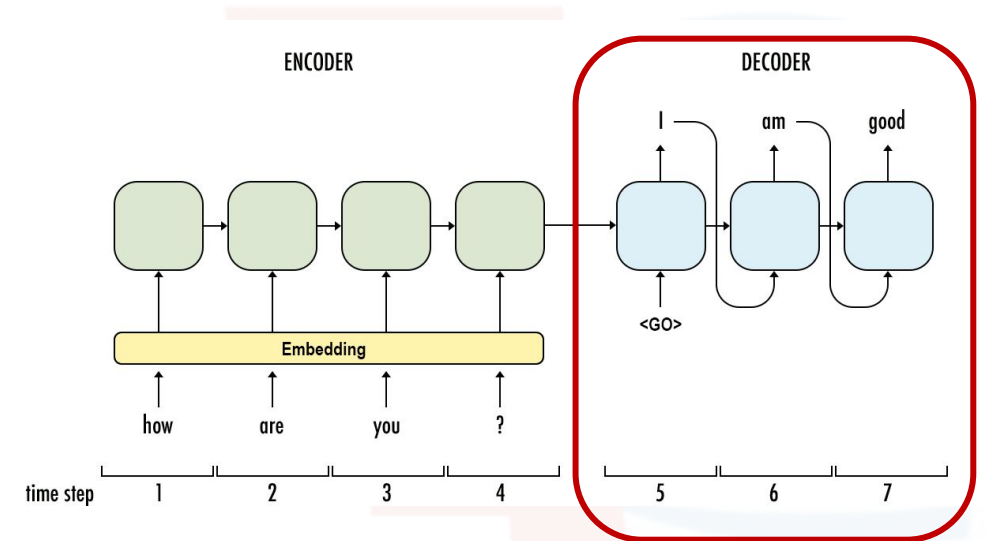


Context Vector

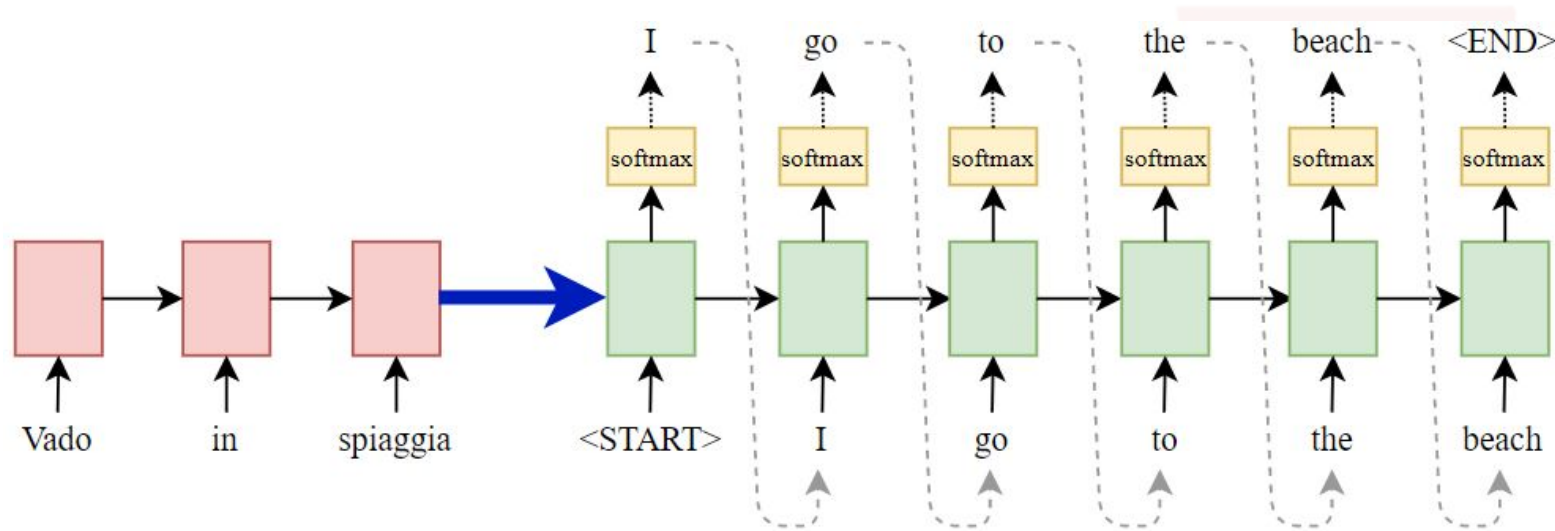


Decoder

- Goal of the decoder:
 - decoding the context vector into the output sequence
 - one element at a time, by “**unfolding**” the LSTM
 - It begins from predicting the first element of the output sequence from a starting token
 - e.g., <SOS> (Start of Sequence token) or <GO> token
 - Then, the next element is predicted by considering the previously predicted token and the long-term memory



Decoder: when does it end generating outputs?

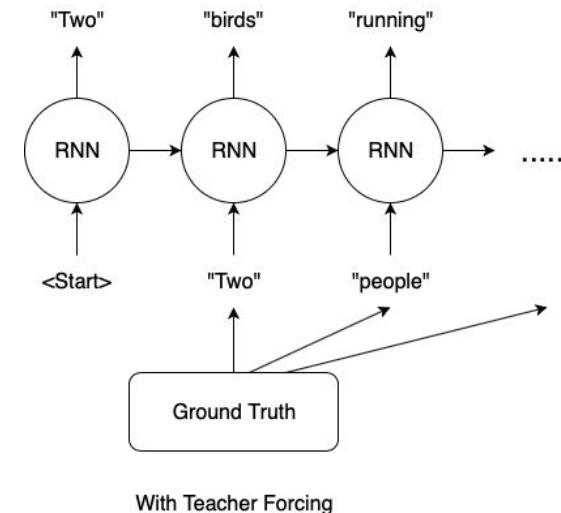
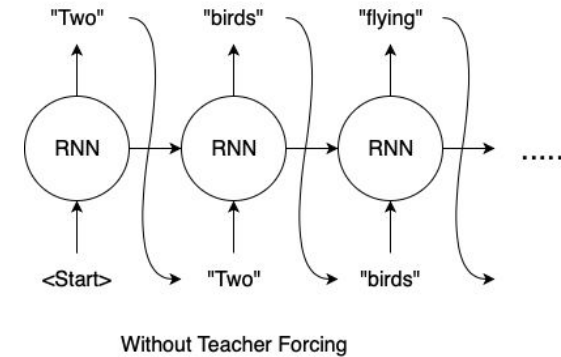


Training

- The encoder receives the input sequence
- The decoder keeps generating one token at a time
 - each generated token is matched with the ground truth!
 - a specific loss is adopted to update weights with backprop
 - the process stops when the ground truth is <EOS> (End of Sequence), even if <EOS> is not generated

Training: Free Running vs Teacher Forcing

- Free Running
 - while decoding, each step of the recurrent network sees the actual output of the previous step
 - realistic, but may be slower in converging since training is affected by error propagation
- Teacher Forcing
 - while decoding, each step of the recurrent network uses the **ground truth** of the previous step
 - faster convergence, but there is a higher risk of overfitting

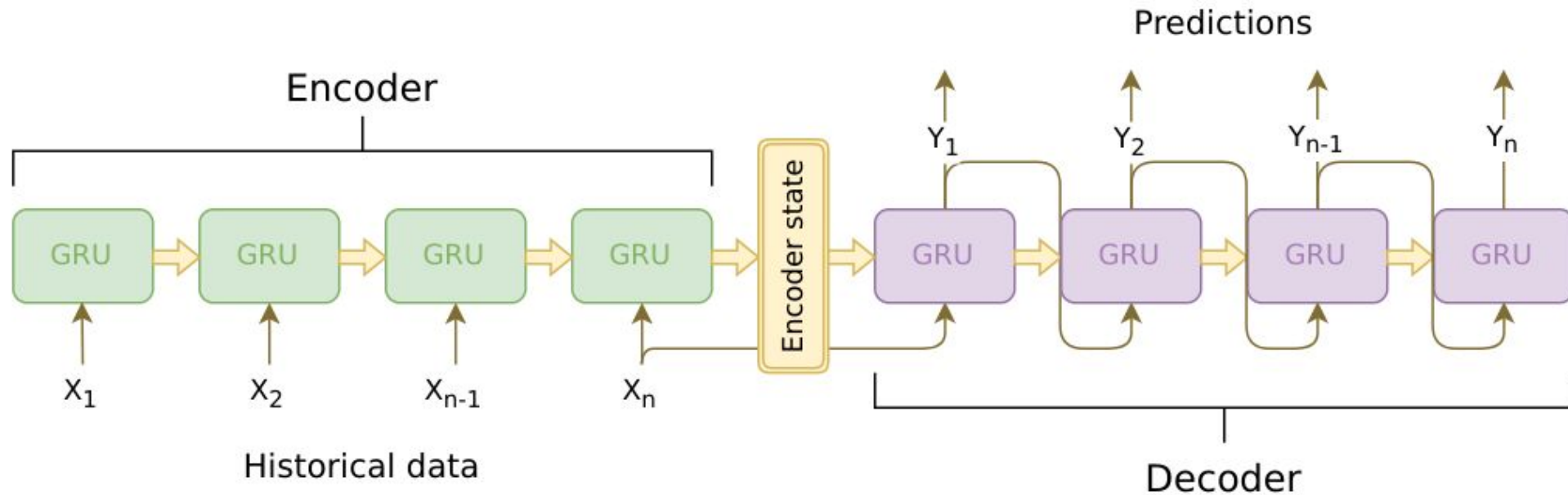


Seq2Seq for Time Series

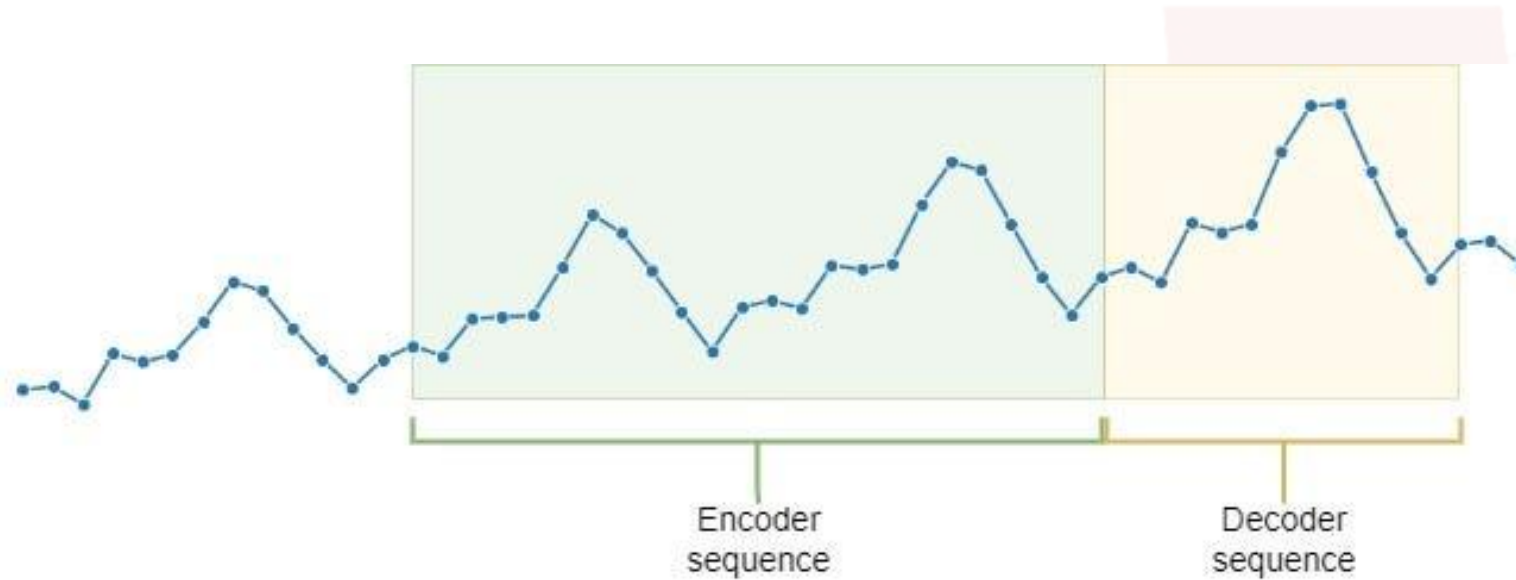
How Seq2Seq changes for Time Series

- Time series are typically represented as a stream of continuous values
 - They do not necessarily need embedding layer to obtain tokens
 - However, it is often a good idea use a linear projection layer to map raw sensor data in a latent space
 - Tokenization may still be needed for time series with categorical data (less common, but possible)
- The main application is usually time series forecasting, hence the $\langle \text{EOS} \rangle$ token is also not needed
 - *e.g., “predict the next 100 values”*
- Instead of starting from the $\langle \text{SOS} \rangle$ token, in time series the decoder starts from the last n samples of the input time series

A seq2seq model for time series



Time Series Forecasting



A sliding window approach for Forecasting



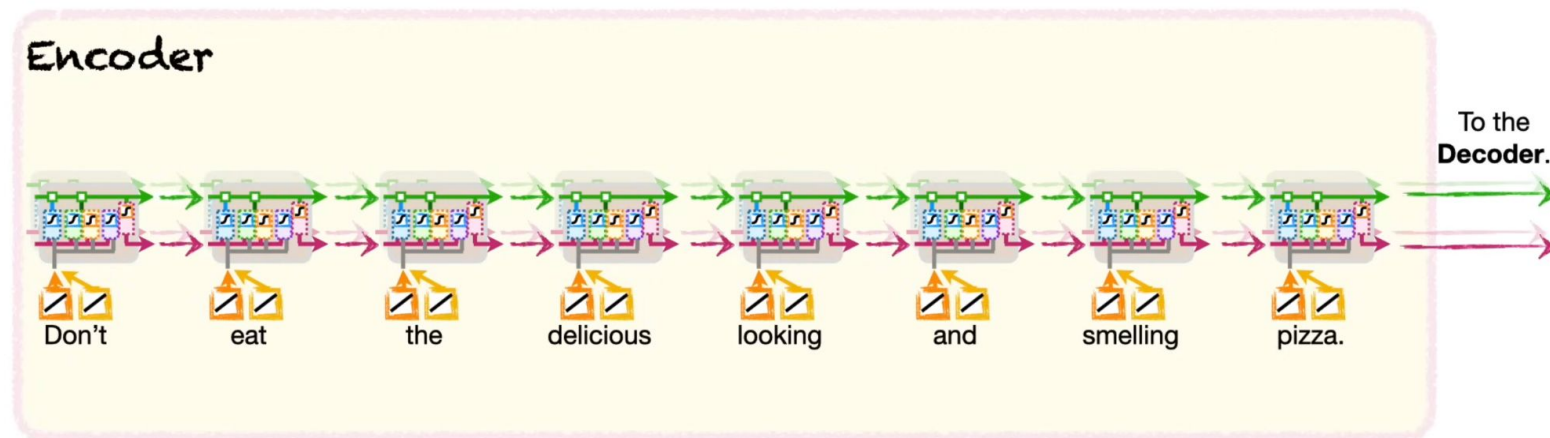
Besides forecasting

- A trained seq2seq model for time series may be used for other tasks. For instance:
 - **Imputation:** using the seq2seq model to reconstruct portions of the time series that are missing
 - **Anomaly Detection:** leveraging the forecasting capabilities of the seq2seq model to check if “predicted” data significantly differs from actual data
 - when observed data differs too much from forecasted data, it may be an anomaly!
 - **Feature representation:** the encoder can be extracted and used alone to generate embeddings
 - **Classification:** the encoder can be extracted and connected to a classification head that it is possible to fine-tune

Attention Mechanism

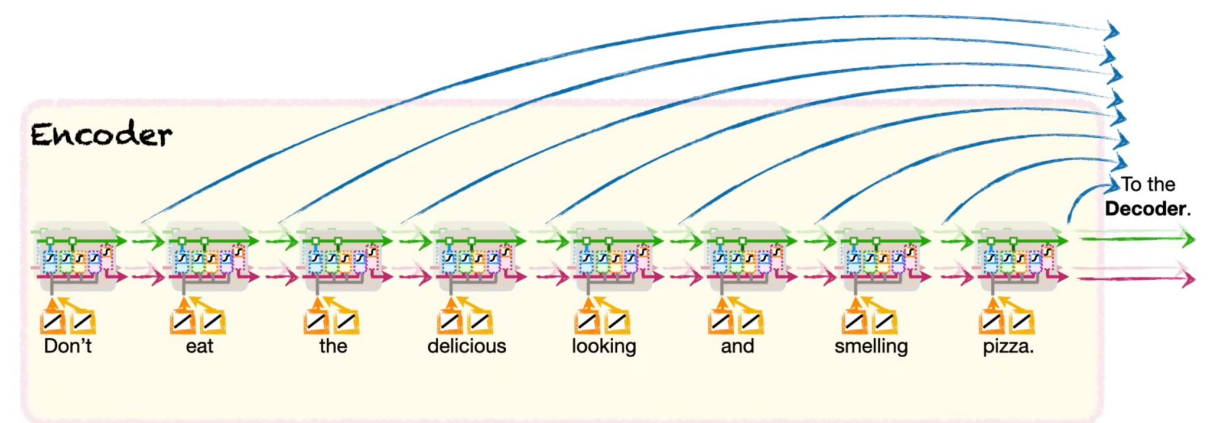
Problems of seq2seq models

- Unfolding LSTMs in the encoder leads to a single context vector
 - fine for short sequences...
 - ...but a problem with larger sequences!
 - LSTMs can still forget “old” elements of the sequence, even using two memories (short-term and long-term)



The Attention Mechanism

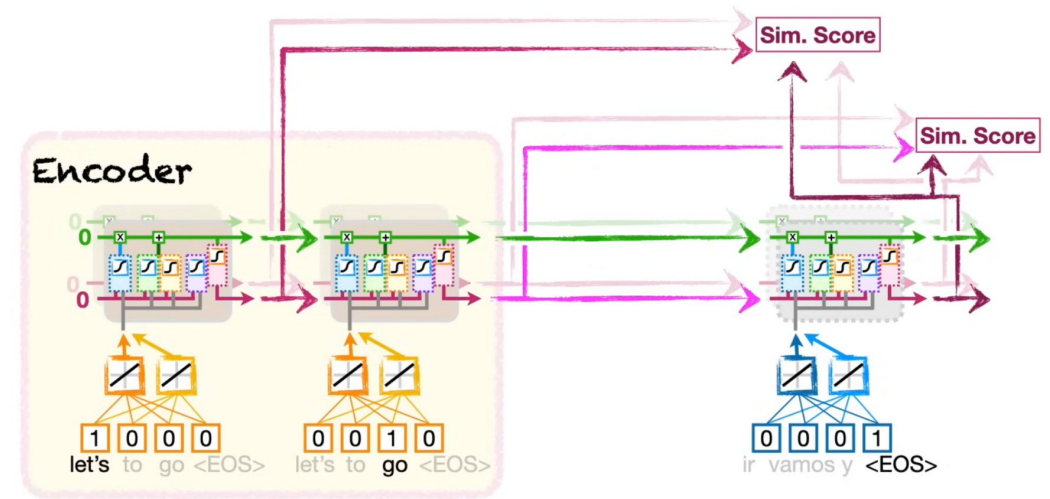
- Intuition: determining how much focus the decoder should place on different parts of the input sequence when generating the current output
- How? Adding many more paths between the encoder and the decoder!
 - one for each input data in the sequence
 - each step of the decoder can access to the output of each step of the encoder
 - similarly to “skip connections”



Main Idea: computing the similarity

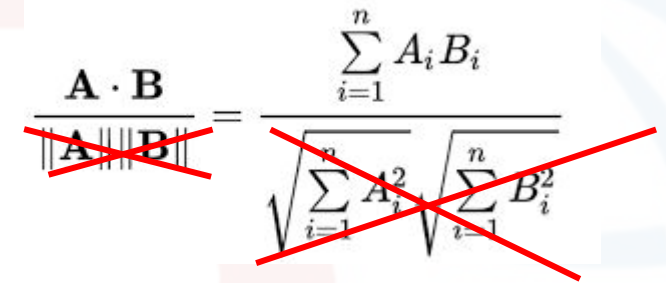
- For each decoder step, it computes the **similarity** of its output with the output of each step of the encoder (using short-term memory)
- Usually done considering the **cosine similarity**

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$



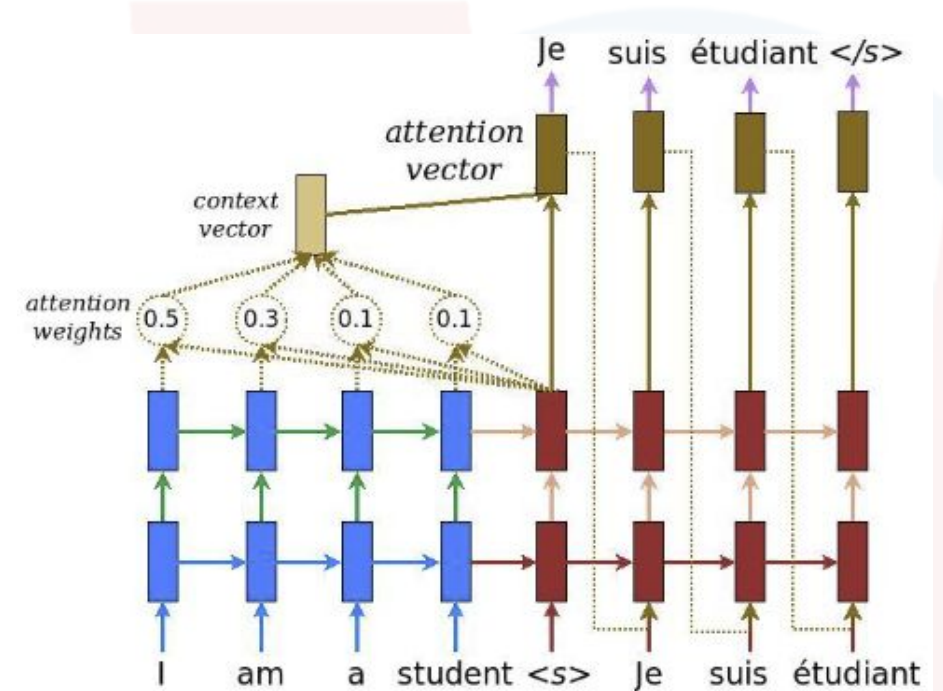
A note about similarity: dot product

- The denominator in the cosine similarity is needed to make sure that the result is in the range $[-1,1]$
- In practice, this is not needed when computing attention and it was observed that the numerator alone (i.e., the dot product) is sufficient
- This also makes the computation faster, improving efficiency!
- Hence, you will often find dot product and not cosine similarity in methods using Attention

$$\frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$


Attention scores

- For each decoder's step, we will have many similarity values
 - one for each encoder's step!
- These values are normalized, usually with a softmax function, to obtain **attention scores** (or **attention weights**)
 - intuitively, each score determines the “importance” of each input element to output the current token
- Attention scores are then combined and used to output the current decoder's step
 - note that, in some implementations, attention scores are processed by a learnable MLP layer before being used by the decoder



Transformers

Limits of Seq2Seq with Attention

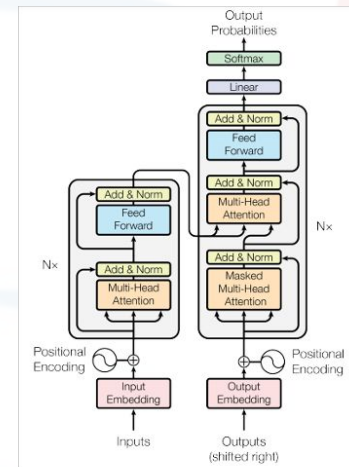
- While Seq2Seq + attention models provided significant improvements with respect to previous models, they still have important limits:
 - using LSTMs still leads to problems related to long-range dependencies
 - slow training, since sequences are processed one element at a time
 - hence, models that are “too deep” may lead to inefficient performances
- Transformers “get rid” of LSTMs!



Transformers Models

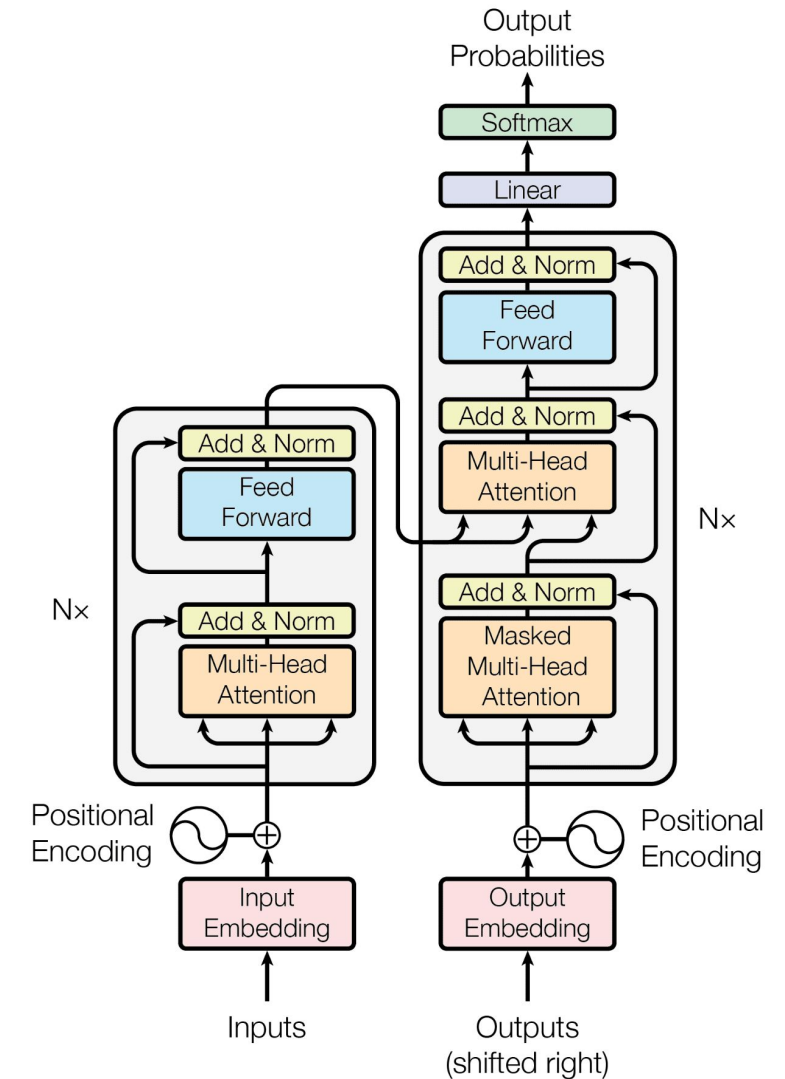
- Improved and advanced version of seq2seq models
- The most common model currently used in NLP applications
 - e.g., Large Language Models
- Very powerful also for time series
- We will not go into deep details (it would require multiple lessons), but we will try to get the intuition

Attention Is All You Need			
Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez* [†] University of Toronto aidan@cs.toronto.edu	Lukas Kaiser* Google Brain lukaszkaizer@google.com	
Illia Polosukhin* [‡] illia.polosukhin@gmail.com			



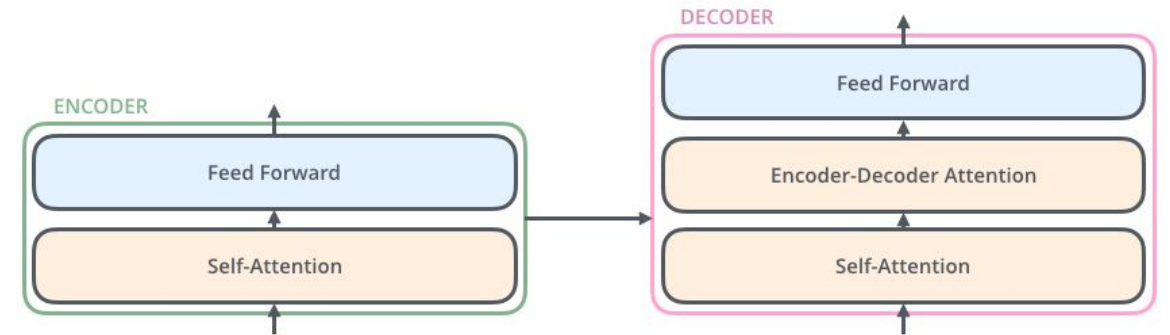
Overall Architecture

- Still an encoder/decoder architecture
- The encoder receives the input sequence and efficiently processes it in parallel
- The decoder takes as input the previous output tokens, the context vector from the encoder, and it outputs one token at a time



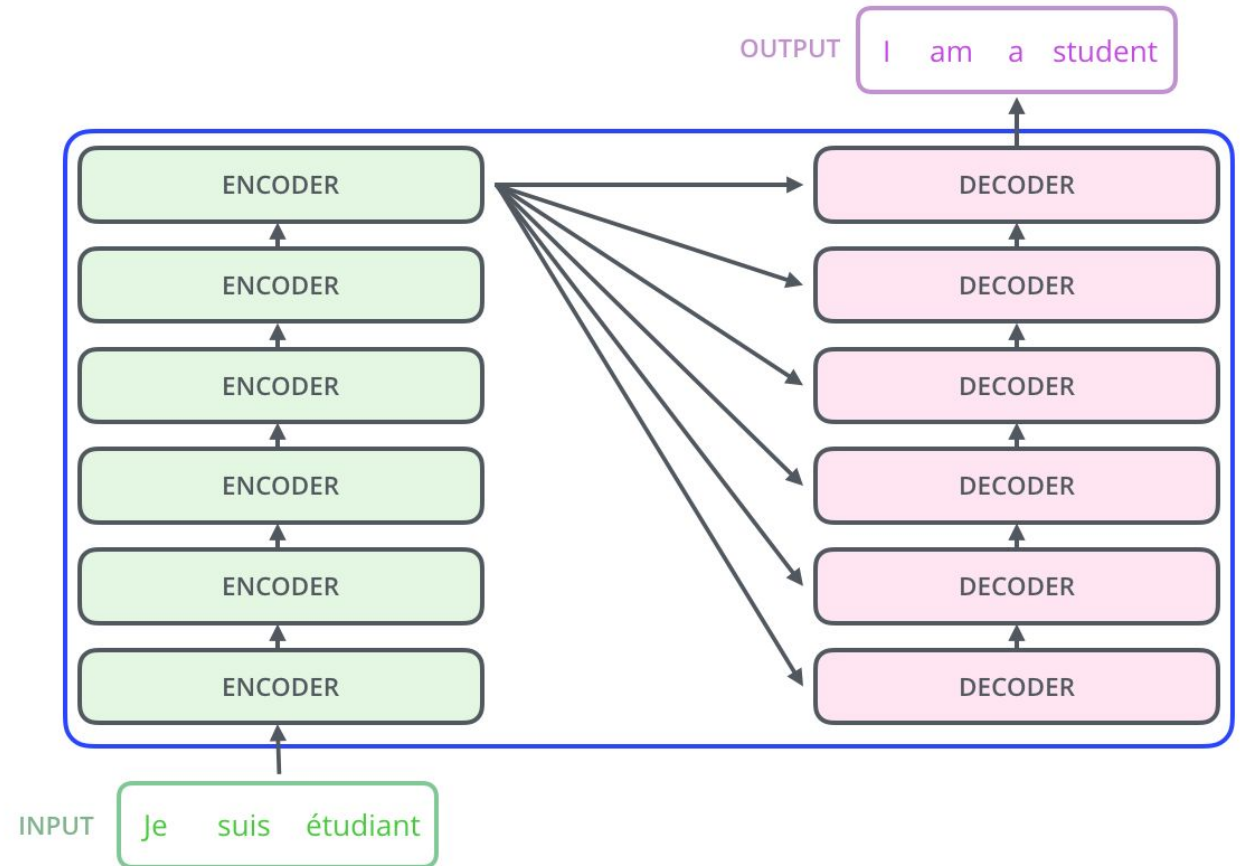
Simplifying a bit

- The self-attention layer is the mechanism (that we will see later) to deal with long-range dependencies in the input sequence
 - it is also used by the decoder, since he receives the previously generated tokens
 - the output of self-attention goes to a feed-forward layer (fully connected layer)
- The Encoder-Decoder attention is the same attention mechanism we learned for seq2seq models!



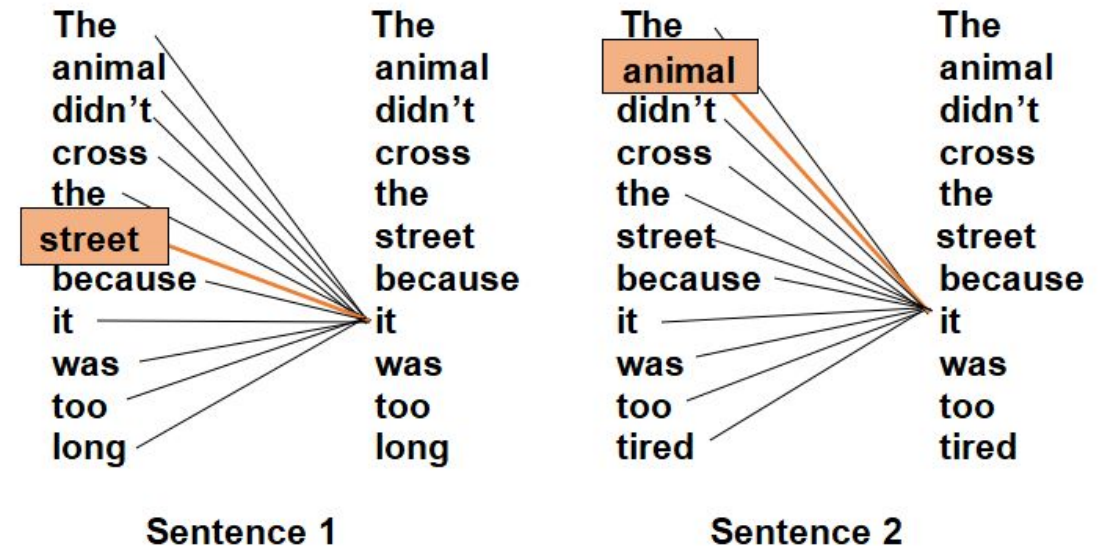
Stacked encoders and decoders

- The encoder component is actually a stack of encoders
 - and similarly the decoder is a stack of the decoders
- Similar concept to CNN models having many Convolutional layers to learn a more representative latent representation



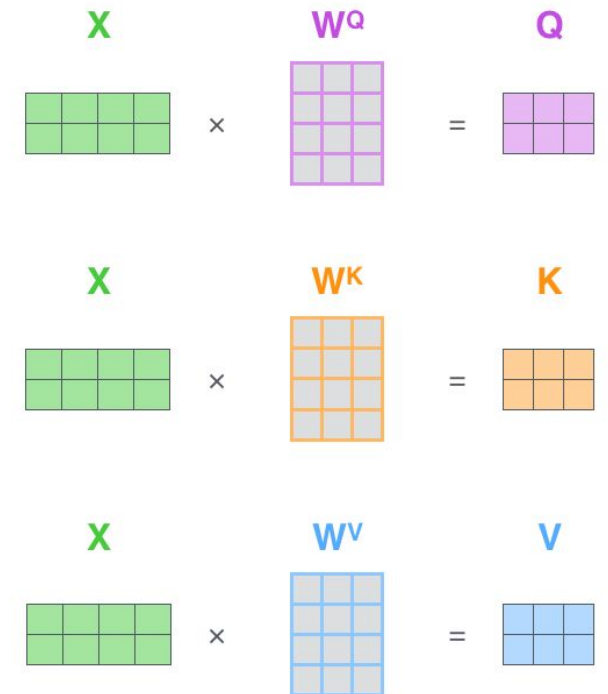
Self-attention

- An effective and efficient way to keep track of long-range dependencies in a sequence
- Intuitively, for each element of a sequence, we compute “attention scores” to capture the most important semantic relationships with every other element
- The computation of self-attention for each element of the sequence can be computed *in parallel!*



Query, Key, and Values

- Each element of the sequence is transformed into three vectors:
 - **Query (Q)**: encodes the token when it is *asking* for information from other tokens in the sequence.
 - **Key (K)**: encodes the token when it is being considered for attention by another token (the token's potential to "respond" to a Query)
 - **Value (V)**: encodes the token in a way that represents the actual information used to update the final output. It will be weighted by the attention score.
- Q, K, and V are obtained with the multiplication of the input with three different weight matrices learned during **training**
- Q and K are used to compute the attention scores, while V is used to compute the weighted combinations



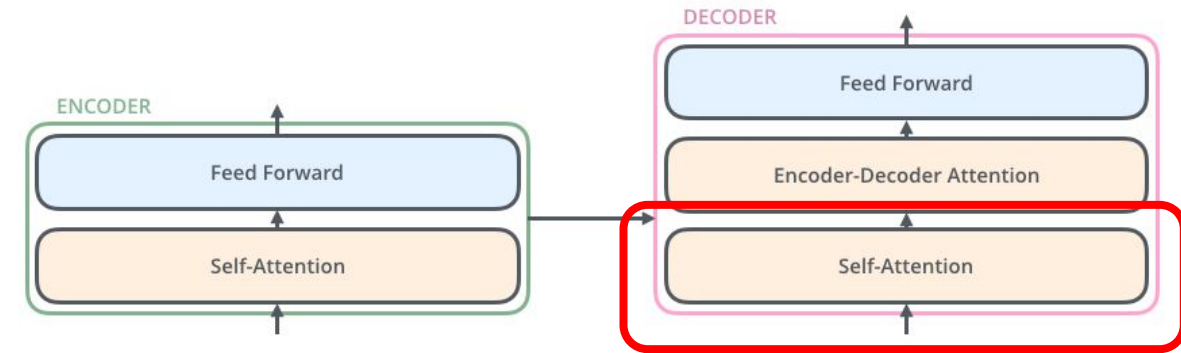
The output of self-attention

- For each element, the attention score is the similarity between Q and K
 - computed using the **dot product** that, as we learned before, it's the **cosine similarity** without the denominator!
- Similarly to what we learned with “classic” attention, all the attention scores are processed by a softmax layer in order to transform them into weights summing to 1
- The weights are then multiplied with V to compute the resulting embedding Z!

$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

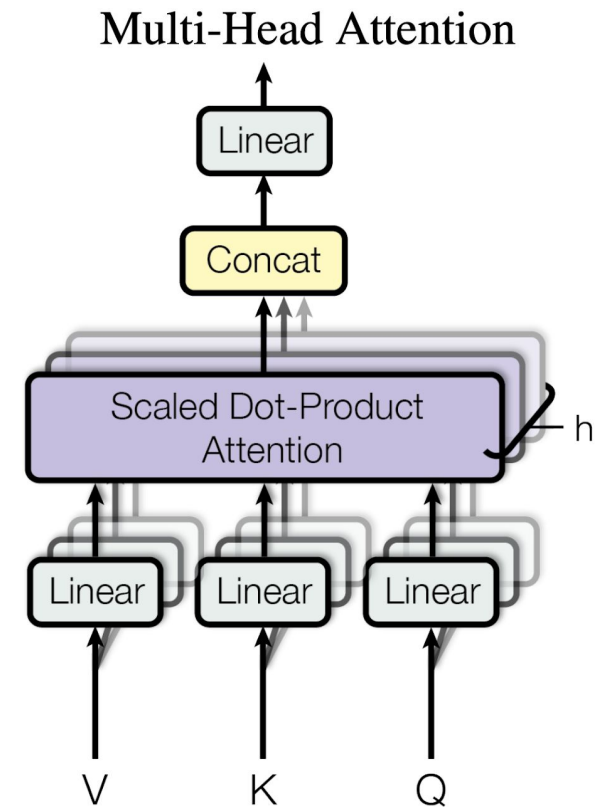
Self-Attention: Encoder VS Decoder

- While the Encoder applies self-attention to the whole input sequence, the Decoder can not look into the future
 - he sees the previously generated tokens and computes self-attention only on those
- Hence, it is also called **masked** self-attention, because future positions are masked



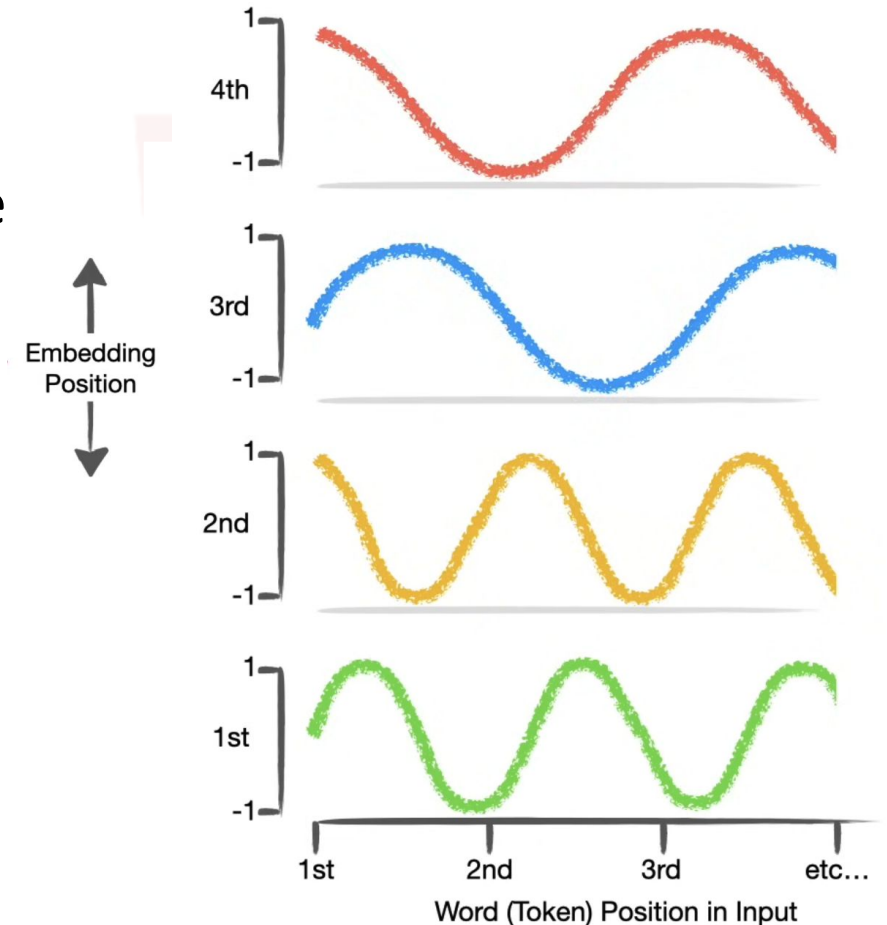
Multi-head Self-Attention

- There are actually multiple self-attention layers working *in parallel*
- They are useful to obtain multiple representation subspaces of the same input and hence improving the model



Positional Encoding

- Since we are still considering a sequence where ***order matters***, how to preserve it without using recurrent networks like LSTMs?
- Positional Encoding makes it possible to include the “order” of the elements in the embedding representation of the input
- It is computed for each dimension of the input by considering the *position* of the token in the input and the *dimension*
 - (supposing each input element is represented as a multi-dimensional vector)
- It is computed using periodic functions (i.e., sine and cosine)



Positional Encoding (cont.)

- For each element in the sequence, we compute the following value for each dimension i :

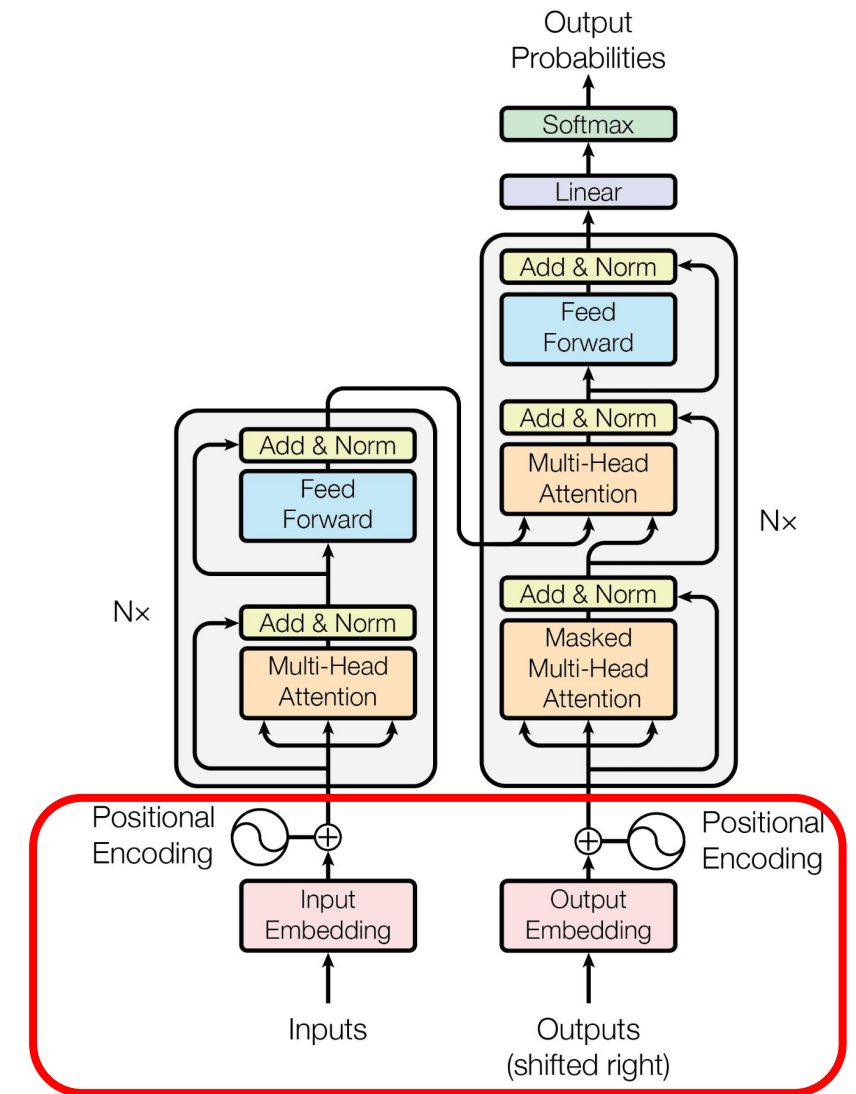
$$PE_{(pos,i)} = \begin{cases} \sin\left(\frac{pos}{10000^{i/d_{model}}}\right) & \text{if } i \text{ is even} \\ \cos\left(\frac{pos}{10000^{(i-1)/d_{model}}}\right) & \text{if } i \text{ is odd} \end{cases}$$

Note that this is only one of the many approaches to compute positional encoding, proposed by the first paper presenting transformers.

The output is a vector of the same size of an element of the sequence.

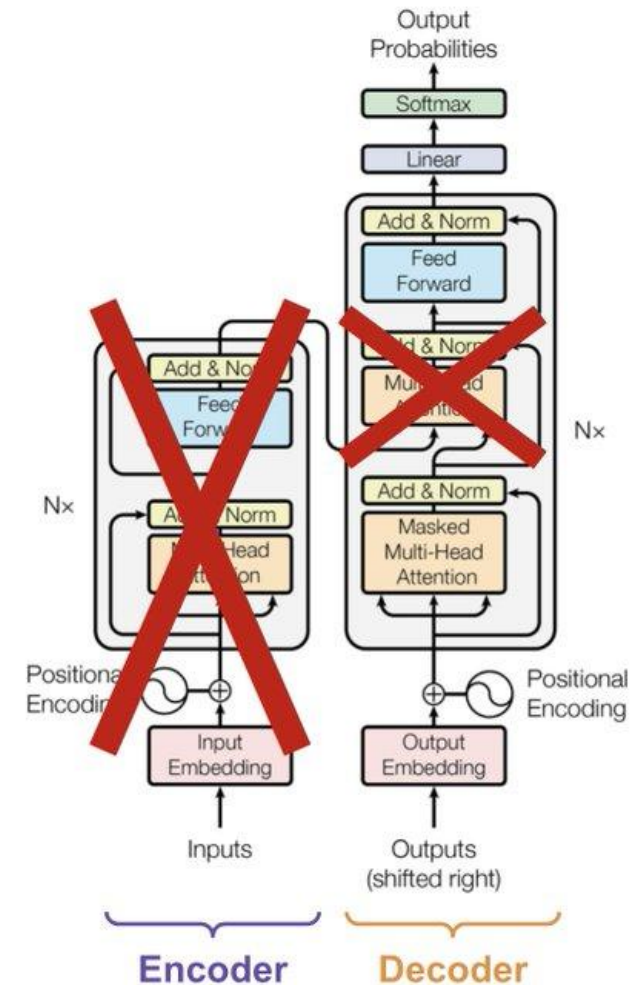
Combining the input with positional embedding

- The embedding representation of the input is summed with positional encoding
 - to obtain an embedding with positional context!
- Positional encoding is needed also for time series
 - in this case, the positional embedding is combined directly with raw multivariate time series or with the output of a linear projection layer



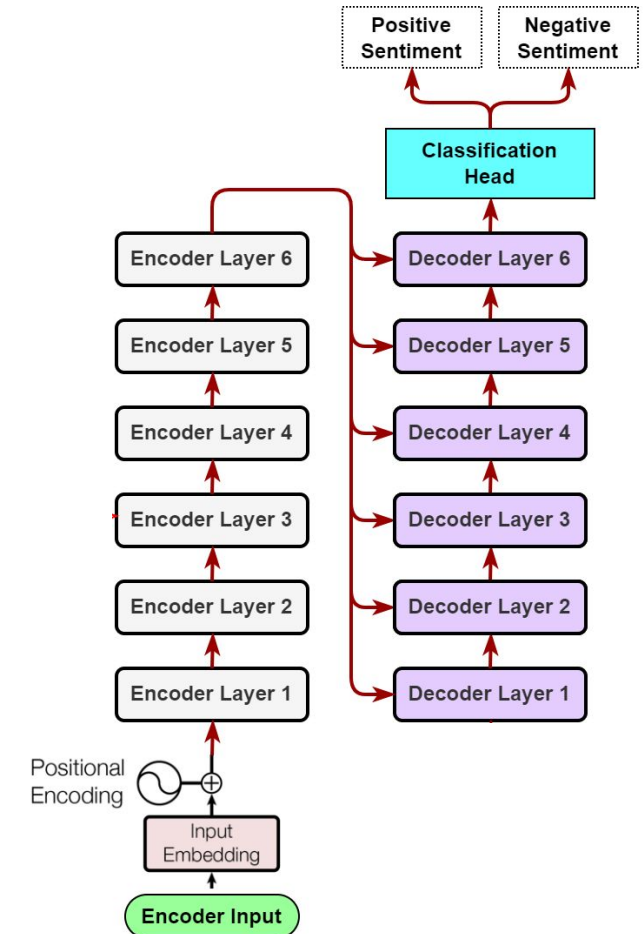
Decoder-only Transformers

- For autoregressive forecasting tasks, it is also possible to leverage only the decoder part of the transformers
 - This is the approach also adopted by GPT models! But it can be impactful for time series as well
- Initially the decoder sees the input sequence, and then it generates one token at a time



Transformers: not only to generate sequences!

- Transformers architectures can be leveraged for other tasks
- For instance, in this example, you can see a “classification head” at the end of the decoder
- Hence, instead of generating one token at a time, the decoder will determine the most likely output class
- Many approaches for time series classification also follow this approach!



Thank you